SE PROJECT: AI AGENT FOR AUTO-GRADING

Complete Development Workflow · April 17 – June 6, 2026

Ángel Pérez Castro · Software Engineering · Gmail Integration Edition

50 days left	Real Gmail API	Gemini Agent	Multi-account

GMAIL INTEGRATION — ARCHITECTURE OVERVIEW

This document reflects the updated architecture where the platform integrates with real Gmail accounts via the Gmail API and OAuth 2.0. Students write their deliverables inside the platform. Upon submission, the system dispatches real emails using a configured Gmail account. In production, multiple Gmail accounts can be linked — one per subject or professor — managed from a Settings panel.

GMAIL INTEGRATION ARCHITECTURE

TEST MODE (single account):

One Gmail account (e.g. se.autograder.test@gmail.com) is configured in Settings. The system sends ALL outgoing emails from this single account using Gmail API + OAuth2. This is the default for development and testing.

PRODUCTION MODE (multi-account):

Multiple Gmail accounts can be registered in the Settings panel (one per subject or per professor). Each account is independently authenticated via OAuth2. When a submission is made under Subject X, the system uses the Gmail account linked to Subject X to send confirmation and feedback emails. A professor can assign or change the linked Gmail account from the Settings UI.

Email triggers (3 events):

1. Student submits a deliverable → System sends confirmation email to student ('Your Deliverable 1 has been received and is being evaluated').
2. AI evaluation completes → System sends feedback email to student with score, criteria breakdown, and full feedback text.
3. Student submits a deliverable → System sends notification email to professor ('Student X has submitted Deliverable 2 for Project Y').

PHASE OVERVIEW — 8 WEEKS AT A GLANCE

#	Dates	Focus	Key Output
1	Apr 17–23	Setup + DB + Auth + Gmail OAuth	**Auth API + Gmail connected**
2	Apr 24–30	Core backend endpoints + email service	**Full API + emails sending**
3	May 1–7	AI evaluation agent (Gemini)	**Auto-grader + email feedback**
4	May 8–14	Frontend: student interface	**Student UI complete**
5	May 15–21	Frontend: professor dashboard	**Professor UI complete**
6	May 22–28	Settings panel + multi-account Gmail	**Multi-account Gmail working**
7	May 29–Jun 4	Testing, deploy & docs	**Live on Railway + Vercel**
8★	Jun 5–6	Final review & submission	**Delivered ✓**

🛠 **PRE-START CHECKLIST** Before Day 1, enable the Gmail API in Google Cloud Console, create OAuth 2.0 credentials (Web Application type), and download the credentials JSON. You will also need the Gemini API key from AI Studio. Have both ready before writing a single line of code.

WEEK 1

Apr 17–23

PHASE 1 — Foundation: Setup, Database, Auth & Gmail OAuth

CONTEXT: This week builds the entire technical skeleton. The key addition compared to a standard project is Gmail OAuth 2.0 — you must get this working early because it affects the Settings model and the email service architecture. Do not skip the Gmail setup even though no emails are sent yet.

▸ FRIDAY–SATURDAY *April 17–18*

◆ Create repo and install all backend dependencies

Create GitHub repo 'se-autograder'. Inside /backend create a Python venv and install: fastapi uvicorn sqlalchemy alembic asyncpg psycopg2-binary python-jose[cryptography] passlib[bcrypt] python-dotenv pydantic-settings google-auth google-auth-oauthlib google-api-python-client google-generativeai. Run pip freeze > requirements.txt immediately.

◆ Design the full database schema including Gmail settings

Define 7 models in /backend/app/models.py: User, Subject, Project, Enrollment, Submission, Evaluation, and — critically — GmailAccount (id, account_email, credentials_json encrypted, subject_id nullable FK, professor_id FK, is_active bool, created_at). The GmailAccount table is the heart of the multi-account system. Each row represents one authenticated Gmail account the system can send from.

◆ Create the .env file with all required variables

DATABASE_URL, SECRET_KEY, ALGORITHM=HS256, ACCESS_TOKEN_EXPIRE_MINUTES=10080, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI=http://localhost:8000/auth/gmail/callback, GEMINI_API_KEY. Store credentials safely — never commit this file.

▸ SUNDAY–MONDAY *April 19–20*

◆ Run Alembic migrations and verify all 7 tables

alembic init migrations, configure env.py, run alembic revision --autogenerate -m 'initial schema', then alembic upgrade head. Connect to PostgreSQL and confirm all 7 tables including gmail_accounts exist.

◆ Implement JWT auth (register, login, token dependency)

Create /backend/app/core/security.py with hash_password, verify_password, create_access_token. Create /backend/app/routers/auth.py with POST /auth/register and POST /auth/login. Create get_current_user dependency in deps.py. Test in Swagger at /docs.

🔒 **SECURITY NOTE** The GmailAccount model stores OAuth2 tokens (access_token, refresh_token, token_expiry) as an encrypted JSON blob in credentials_json. Use the cryptography package (Fernet) to encrypt this field at rest. The encryption key goes in .env as FERNET_KEY. Add this package to requirements now: pip install cryptography.

▸ TUESDAY–WEDNESDAY *April 21–22*

◆ Implement the Gmail OAuth 2.0 authorization flow

Create /backend/app/routers/gmail_auth.py. Implement GET /auth/gmail/authorize: uses google-auth-oauthlib to build an authorization URL with scopes ['https://www.googleapis.com/auth/gmail.send'] and redirects the professor to Google's consent screen. Implement GET /auth/gmail/callback: Google redirects here with an auth code. Exchange it for tokens, encrypt the credentials JSON with Fernet, save a new GmailAccount row. This is the one-time setup a professor does to connect their Gmail.

◆ Build the email service module (skeleton)

Create /backend/app/services/email_service.py. Implement get_gmail_service(account_email, db): loads the GmailAccount row, decrypts the credentials JSON, builds a google.oauth2.credentials.Credentials object, refreshes the token if expired, and returns an authenticated Gmail API service object. Implement send_email(to,

subject, body_html, gmail_account_email, db): calls get_gmail_service and uses the Gmail API's messages.send with a base64-encoded MIME message. Test this function standalone by sending a test email to yourself.

▸ THURSDAY *April 23*

- **Wire up the GmailAccount settings endpoints**
  Create /backend/app/routers/settings.py. Implement: GET /settings/gmail-accounts (professor only — list all Gmail accounts linked to them), DELETE /settings/gmail-accounts/{id} (remove a linked account), POST /settings/gmail-accounts/{id}/set-default (mark one account as the default sender for a subject). These endpoints are what the Settings UI will call.
- **Write integration tests for auth and Gmail flow**
  In /backend/tests/test_auth.py test register, login, bad credentials. In /backend/tests/test_gmail.py test that get_gmail_service correctly decrypts credentials and refreshes expired tokens (use mocked credentials). Push all code to GitHub.

✓ END-OF-WEEK DELIVERABLE: WEEK 1: FastAPI running with 7-table PostgreSQL schema. JWT auth working. Gmail OAuth2 flow implemented — professor can authorize a Gmail account and the system stores encrypted credentials. Email service can send a real email via the Gmail API.

WEEK 2
Apr 24–30

PHASE 2 — Core Backend: All Endpoints + Email Dispatch Service

CONTEXT: This week you build all business logic endpoints AND wire up the three email dispatch events. By end of week, submitting a deliverable via the API should trigger two real Gmail emails: a confirmation to the student and a notification to the professor.

▸ FRIDAY–SATURDAY *April 24–25*

- **Implement Subject and Project CRUD endpoints**
  POST/GET/PUT/DELETE /subjects (professor only, auto-assigns owner). POST/GET/PUT/DELETE /subjects/{id}/projects. Each project has: name, description, topic, and a gmail_account_id FK (nullable — which Gmail account handles emails for this project). Add a PATCH /projects/{id}/gmail-account endpoint so professors can assign a Gmail account to a project from the Settings panel.
- **Implement Enrollment and the sequential submission logic**
  POST /enrollments (student enrolls in project). POST /submissions with full sequential lock: deliverable N requires deliverable N-1 to have an Evaluation row. Implement check_submission_allowed(enrollment_id, deliverable_number, db) as a reusable function returning (allowed: bool, reason: str).

▸ SUNDAY–MONDAY *April 26–27*

- **Implement the email resolver — which Gmail account to use**
  Create /backend/app/services/email_resolver.py. Implement resolve_sender_account(project_id, db) -> GmailAccount: (1) Check if the project has a gmail_account_id set — use that. (2) If not, check if the project's subject has a default Gmail account — use that. (3) If not, fall back to the professor's personal linked Gmail account. (4) If none configured, raise a clear error: 'No Gmail account configured for this project — please configure one in Settings.' This resolver is called every time an email needs to be sent.
- **Wire the three email triggers into the submission endpoint**
  In POST /submissions, after saving the Submission to DB and before returning the response, add these three background tasks: (1) send_submission_confirmation(submission, student, project, db) — sends 'Your Deliverable X has been received' to the student's email. (2) send_professor_notification(submission, student, project, professor, db) — sends 'Student Y submitted Deliverable X for Project Z' to the professor. (3) trigger_ai_evaluation(submission.id, db) — asynchronous, kicks off the Gemini evaluation (built in week 3). All three use BackgroundTasks so the API response returns immediately.

✉ EMAIL TEMPLATES Email templates: create /backend/app/services/email_templates.py with three functions that return HTML strings: confirmation_email(student_name, deliverable_num, project_name, deadline_next),

professor_notification_email(student_name, deliverable_num, project_name, submission_preview), and feedback_email(student_name, deliverable_num, score, criteria_breakdown, feedback_text). Well-formatted HTML emails make the project look professional.

▸ TUESDAY–WEDNESDAY *April 28–29*

- **Implement all GET endpoints for student and professor views**
  GET /enrollments (student's own list with progress). GET /enrollments/{id}/submissions (all submissions + evaluations for one enrollment). GET /projects/{id}/enrollments (professor: all students in a project). GET /submissions/{id}/evaluation (returns evaluation or {status: pending}). GET /settings/gmail-accounts (professor's linked accounts). These are the endpoints the frontend will call most.
- **Implement the professor override endpoint**
  POST /evaluations/{id}/override: professor sends override_score (0-100) and override_comment (required). Verify professor owns the project. Update Evaluation. Then send a feedback_email to the student with the overridden score and professor comment (using the same email service and resolver).

▸ THURSDAY *April 29*

- **Test real email delivery end-to-end**
  Using Postman: register a professor, authorize a Gmail account via OAuth, create a subject and project, assign the Gmail account to the project, register a student, enroll, submit deliverable 1. Check two inboxes: the student's should receive a confirmation email, the professor's should receive a notification email. If tokens are expired, verify the refresh logic works automatically.
- **Handle the 'no Gmail account configured' gracefully**
  When the email resolver cannot find a Gmail account, the system must NOT crash and must NOT silently swallow the error. Log the error clearly, save a flag on the Submission (email_sent: bool), and continue. The professor dashboard will show a warning badge 'Email not sent — no Gmail configured' for affected projects.

⚠ GMAIL QUOTA Gmail API has a sending quota of 500 emails/day for free accounts. For a class of 30 students submitting 4 deliverables each, that is 30×4×3 = 360 emails minimum. You are within the limit for testing, but document this constraint clearly for production.

✓ END-OF-WEEK DELIVERABLE: WEEK 2: Complete REST API. Three email events firing real Gmail emails. Email resolver selecting the correct account per project. Professor notification and student confirmation tested in real inboxes.

WEEK 3
May 1–7

PHASE 3 — AI Evaluation Agent + Email Feedback Delivery

CONTEXT: This week you build the Gemini evaluation agent and complete the third email trigger: when the AI finishes grading, it sends the feedback email to the student. The agent must produce structured output (JSON) and the feedback email must be beautifully formatted HTML.

▸ FRIDAY–SATURDAY *May 1–2*

- **Set up Gemini and design the 4 evaluation prompts**
  pip install google-generativeai. Add GEMINI_API_KEY to .env. Create /backend/app/ai/prompts.py with 4 system prompt templates. Each receives: project_topic, deliverable_content, and previous_submissions list. Each prompt ends with: 'Respond ONLY with valid JSON: {score: int 0-100, criteria: {name: score}, feedback: string min 150 words}'. Test all 4 prompts manually in Google AI Studio before writing any Python code.
- **Implement the 4 evaluator functions**
  Create /backend/app/ai/evaluator.py. D1 criteria: Research depth (25), Source quality (20), Motivation clarity (30), Writing structure (25). D2 criteria: REQ format correctness (20), No ambiguity (25), Completeness (30), Traceability to D1 (25). D3 criteria: Question quality (35), Coverage of new use cases (35), Variety and depth (30).

D4 criteria: Integration of D3 findings (40), Consistency with D1+D2 (30), Document maturity (30). Each function validates the JSON output and raises on malformed response.

▸ SUNDAY–MONDAY *May 3–4*

◆ Build the evaluation dispatcher with feedback email trigger

Create /backend/app/ai/dispatcher.py. Implement run_evaluation(submission_id, db): fetch submission + all previous submissions for context, call the correct evaluator, save Evaluation row, THEN call send_feedback_email(student, submission, evaluation, project, db). The feedback email is the most important email in the system — it is what the student actually cares about. Wrap everything in try/except: on Gemini failure, save a fallback evaluation and still send an email saying 'Evaluation failed — your professor will review manually'.

◆ Design the feedback email template

In email_templates.py, implement feedback_email(). This HTML email must show: a large score badge (green/orange/red based on score), a visual criteria breakdown table (each criterion as a row with score), the full feedback text in a readable block, and a 'Log in to view details' button linking to the platform. This email represents the system to the student — make it look professional.

✉ FEEDBACK EMAIL The feedback email is sent from the same Gmail account configured for the project (resolved via email_resolver). If the professor has overridden the score, send a second email with a 'Score updated by your professor' badge and the new score + professor comment.

▸ TUESDAY–WEDNESDAY *May 5–6*

◆ Prompt quality testing and iteration

Create /backend/tests/test_evaluator.py with sample submissions of varying quality per deliverable type. Run all 12+ test evaluations. Inspect: Are scores proportional to quality? Is the feedback specific or generic? Does D4 actually reference what was in D3? Adjust prompts until the agent behaves like a knowledgeable, fair grader. This is real prompt engineering — budget 4–6 hours.

◆ Add the email_sent audit log

Add a new model EmailLog (id, submission_id FK, email_type enum: confirmation/professor_notification/feedback/override_feedback, recipient_email, gmail_account_used, sent_at, error_message nullable). Log every email attempt, success or failure. This gives professors visibility into what was sent and when, and helps debug delivery issues.

▸ THURSDAY *May 7*

◆ Full integration test: submit → evaluate → email

Using real accounts: submit a deliverable, verify the confirmation email arrives within seconds, wait for Gemini evaluation (10-30 seconds), verify the feedback email arrives with a real score and formatted feedback. Test the failure path: temporarily revoke the Gmail OAuth token and verify the fallback error handling works without crashing the API.

✓ END-OF-WEEK DELIVERABLE: WEEK 3: AI agent evaluating all 4 deliverable types. Feedback emails delivered to students via real Gmail. EmailLog table recording all activity. All 3 email triggers verified end-to-end.

WEEK 4
May 8–14

PHASE 4 — Frontend: Student Interface

CONTEXT: Backend is complete and sending real emails. This week you build the student-facing React app. Students write deliverables here, see their scores in the platform, AND receive the results by email. The platform and email are complementary: the platform is the workspace, the email is the notification.

▸ FRIDAY–SATURDAY *May 8–9*

◆ Bootstrap React + Vite + Tailwind and set up routing

npm create vite@latest . -- --template react. npm install -D tailwindcss postcss autoprefixer && npx tailwindcss init -p. npm install axios react-router-dom. Create Axios client with JWT interceptor. Create AuthContext with login/logout. Routes: /login, /register, /dashboard, /browse, /workspace/:enrollmentId.

- **Build Login, Register and PrivateRoute**
  LoginPage: email + password form, calls POST /auth/login, stores JWT, redirects to /dashboard. RegisterPage: adds a Role selector (student/professor). PrivateRoute: wraps protected routes and redirects to /login if no token. ProfessorRoute: redirects students trying to access professor pages.

▸ SUNDAY–MONDAY *May 10–11*

- **Build the subject browser and enrollment flow**
  BrowsePage: fetches GET /subjects, displays subject cards, expands to show projects with topic and description. Each project has an 'Enroll' button. On click: POST /enrollments, then redirect to /workspace/:enrollmentId. Show a success toast: 'Enrolled! Check your email for confirmation.'
- **Build the Workspace page — the 4-deliverable timeline**
  WorkspacePage: fetches GET /enrollments/{id}/submissions. Shows a vertical 4-step timeline. Each step shows: deliverable number and name, status (locked/open/submitted/graded), deadline countdown, and score badge if graded. The active step has a prominent 'Write & Submit' button. Locked steps show a padlock icon with reason ('Submit and pass Deliverable 1 first').

▸ TUESDAY–WEDNESDAY *May 12–13*

- **Build the submission form with email confirmation hint**
  SubmissionForm: a full-page textarea with the deliverable instructions at top and a live word count. On submit: POST /submissions. Show immediate feedback: a spinner with 'Submitting... A confirmation email will be sent to your inbox.' Then show: 'Submitted! The AI is now evaluating your work. You will receive your score and feedback by email within 1–2 minutes.'
- **Build the in-platform feedback view**
  FeedbackCard component: score badge (circular, color-coded), criteria breakdown as a table with progress bars, full feedback text. This appears inside WorkspacePage when an evaluation exists. Add a note: 'This feedback was also sent to your email.' If a professor override exists, show both scores: 'AI Score: 72 → Professor Score: 85' with the override comment.

▸ THURSDAY *May 13*

- **Build the student dashboard (my projects overview)**
  DashboardPage: list of all enrollments as cards showing project name, subject, current progress (e.g. 'Step 2 of 4'), latest score, and next deadline. A top banner shows: 'You have N deliverables due this week'. Clicking a card goes to WorkspacePage.
- **Manual end-to-end test of the full student flow**
  Run real backend + real Gemini + real Gmail. In browser: register student → browse → enroll → submit D1 → see spinner → see feedback in platform → check email inbox (confirmation + feedback should both arrive). Test with a real Gmail account as the student's registered email.

✓ END-OF-WEEK DELIVERABLE: WEEK 4: Complete student React app. Login, browse, enroll, submit, and view feedback all working. Email delivery verified from the student's perspective.

WEEK 5
May 15–21

PHASE 5 — Frontend: Professor Dashboard + Settings Panel

CONTEXT: This week you build the professor interface AND the Settings panel for Gmail account management. The Settings panel is the unique feature of this project — it lets professors connect multiple Gmail accounts and assign them to subjects/projects.

▸ FRIDAY–SATURDAY *May 15–16*

◆ Build professor subject and project management

ManagePage: list professor's subjects, create/edit/delete subjects via forms, create/edit/delete projects. Each project card shows the currently assigned Gmail account (or a warning if none assigned). An 'Assign Gmail Account' dropdown lets the professor select from their connected Gmail accounts.

◆ Build the professor student progress dashboard

ProfessorDashboard: a table of all enrollments across all professor's projects. Columns: Student, Project, Progress (2/4), Latest Score, Last Activity, Email Status (shows warning icon if any email failed to send), Status (active/overdue). Filterable by subject and project.

▸ SUNDAY–MONDAY *May 17–18*

◆ Build the student detail view for professors

StudentDetailPage: all 4 deliverable cards for one enrollment. Each submitted deliverable shows: submission text, AI evaluation (score + criteria + feedback), email delivery status (confirmation sent ✓, feedback sent ✓). An 'Override Score' inline panel: number input + required comment. On save: PATCH /evaluations/{id}/override → shows success + 'Override feedback email sent to student'.

◆ Build the Gmail Settings panel — the centrepiece of production mode

SettingsPage: a dedicated page at /professor/settings. Section 'Connected Gmail Accounts': list of all linked accounts with email address, connection status (active/token expired), and a 'Disconnect' button. A large 'Connect New Gmail Account' button triggers the OAuth flow: opens a popup to /auth/gmail/authorize. After the professor grants access, the callback saves the new GmailAccount and the settings page refreshes. Section 'Account Assignments': a table where each subject/project can have a Gmail account selected from a dropdown.

✉ OAUTH POPUP PATTERN The OAuth popup approach for connecting Gmail accounts is important UX. The main app should not navigate away during the OAuth flow. Use window.open() to open the authorization URL in a popup. The popup closes after the callback and sends a postMessage to the parent window to trigger a refresh of the connected accounts list.

▸ TUESDAY–WEDNESDAY *May 19–20*

◆ Build the email audit log view for professors

In the student detail view, add an 'Email History' expandable section showing the EmailLog entries for that student's submissions: type (confirmation/feedback/override), sent timestamp, which Gmail account sent it, and any error message. This gives professors confidence that communications were actually delivered.

◆ Build the test email sender in settings

In the SettingsPage, next to each connected Gmail account, add a 'Send Test Email' button. Clicking it calls POST /settings/gmail-accounts/{id}/test — the backend sends a simple test email from that account to the professor's own registered email. This lets professors verify a newly connected account works before assigning it to a live project.

▸ THURSDAY *May 21*

◆ Complete professor routing and navigation

Professor routes: /professor/dashboard, /professor/manage, /professor/settings, /professor/student/:enrollmentId. Top navbar for professors: Dashboard, Manage Subjects, Settings (with a gear icon), and name + logout. Make sure role-based redirects work: professors always land on /professor/dashboard after login.

◆ Create seed data script with Gmail-aware data

Create /backend/scripts/seed.py. Creates: 1 professor (with a mock GmailAccount row pointing to your test Gmail), 5 students, 3 subjects, 2 projects per subject. Each project is assigned the test Gmail account. Run seed on local DB and verify the professor dashboard shows populated data with email statuses.

✓ END-OF-WEEK DELIVERABLE: WEEK 5: Complete professor UI including student progress, score override, and the Gmail Settings panel. Professor can connect multiple Gmail accounts, assign them to projects, and send test emails.

WEEK 6
May 22–28

PHASE 6 — Multi-Account Testing, Integration & Bug Fixing

CONTEXT: The app is functionally complete. This week is dedicated to testing the multi-account Gmail scenario specifically, plus general integration testing and bug fixing. By end of week the system must handle 2+ Gmail accounts without mixing them up.

▸ FRIDAY–SATURDAY *May 22–23*

- **Multi-account Gmail scenario test**
  Connect 2 different Gmail accounts to the test professor. Assign Account A to Subject 1 and Account B to Subject 2. Create a student and enroll them in one project from each subject. Submit deliverable 1 in both. Verify: Subject 1 emails come from Account A, Subject 2 emails come from Account B. Check both sent inboxes and both student inboxes. This is the core production scenario.
- **Test token refresh for expired Gmail tokens**
  Manually expire a Gmail token in the database (set token_expiry to a past timestamp). Trigger an email send. Verify the system automatically refreshes the token using the refresh_token, updates the database, and sends the email successfully — without any manual intervention.

▸ SUNDAY–MONDAY *May 24–25*

- **Test the fallback chain in email_resolver**
  Scenario 1: project has a gmail_account_id set → correct account used. Scenario 2: project has no account, subject has a default → subject account used. Scenario 3: neither project nor subject has an account, professor has a personal account → professor account used. Scenario 4: no account anywhere → clear error returned to API, email_sent=False, professor dashboard shows warning. Test all 4 scenarios.
- **Backend security and permission audit**
  Verify: students cannot access professor endpoints, professors cannot override evaluations outside their projects, students cannot view other students' submissions, Gmail credentials are stored encrypted and never returned in any API response. Fix any issues found.

▸ TUESDAY–WEDNESDAY *May 26–27*

- **Frontend error handling and loading state audit**
  Every async action must have a loading state and error state. Specifically for email-related actions: if 'Connect Gmail' OAuth fails (popup closed), show a helpful message. If 'Send Test Email' fails, show the reason. If a submission is saved but email fails, show 'Submission saved. Email delivery failed — check Settings.' without alarming the student about the underlying error.
- **Full regression test — complete journey for both roles**
  Reset the database with the seed script. As professor: connect a Gmail account, create a subject and project, assign the account. As student: enroll, submit all 4 deliverables one by one, receive all confirmation and feedback emails. As professor: view the dashboard, check email logs, override D2 score, verify student gets override email. Verify zero errors throughout.

✓ END-OF-WEEK DELIVERABLE: WEEK 6: Multi-account Gmail verified. Token refresh working. Fallback chain tested in all 4 scenarios. Full regression passed. Email audit log complete.

WEEK 7
May 29–Jun 4

PHASE 7 — Deploy, Documentation & Final Polish

CONTEXT: Deploy to production and prepare everything for submission. The Gmail OAuth flow requires specific configuration in Google Cloud Console for production URLs — this is a common stumbling block that costs hours if left to the last day.

▸ FRIDAY–SATURDAY *May 29–30*

- **Deploy PostgreSQL and backend on Railway**
  Create Railway project, provision PostgreSQL, copy DATABASE_URL. Deploy FastAPI service from GitHub. Set all environment variables: DATABASE_URL, SECRET_KEY, GEMINI_API_KEY, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, FERNET_KEY. Update GOOGLE_REDIRECT_URI to https://your-backend.railway.app/auth/gmail/callback. Run alembic upgrade head against production DB.
- **Update Google Cloud Console for production**
  In Google Cloud Console, add the Railway backend URL to Authorized Redirect URIs. Add the Vercel frontend URL to Authorized JavaScript Origins. Without these, the Gmail OAuth flow will fail in production with a redirect_uri_mismatch error. Test the OAuth flow on the production URL before deploying the frontend.

⚠ **OAUTH REDIRECT MISMATCH** The most common deployment failure: GOOGLE_REDIRECT_URI in the backend .env must exactly match the URI registered in Google Cloud Console — including https vs http and trailing slashes. Double-check this before debugging anything else.

▸ SUNDAY–MONDAY *May 31–Jun 1*

- **Deploy React frontend on Vercel**
  Import /frontend from GitHub into Vercel. Set VITE_API_URL to the Railway backend URL. After deployment, test the complete login flow on the production URL. Test connecting a Gmail account via the Settings panel on production — the OAuth popup must work on the live domain.
- **Write the project README**
  Root /README.md: project description, Gmail integration architecture diagram (ASCII), local setup instructions (step by step, under 15 steps), environment variables reference, how to connect a Gmail account (step by step), how to run tests, live demo URL, demo video link.

▸ TUESDAY–WEDNESDAY *Jun 2–3*

- **Production smoke test — all features on live URL**
  Test on production: register professor → connect Gmail in Settings → create subject and project → assign Gmail → register student → enroll → submit D1 → confirmation email arrives → feedback email arrives with score → professor views dashboard → override score → student gets override email. Zero errors allowed.
- **Record the demo video**
  Record a 4–5 minute Loom video showing: the Settings panel connecting a Gmail account, a student submitting a deliverable, the confirmation email arriving in the inbox, the AI feedback email arriving, and the professor dashboard. This video is your backup for the presentation.

▸ THURSDAY *Jun 4*

- **Final UI polish**
  Check every page for visual consistency. Ensure the Gmail status indicators (connected/expired/warning) are clear and well-styled. Make sure email-related UI copy is clear to non-technical users ('Your results will be emailed to angel@gmail.com' is better than 'Email dispatch queued').

✓ **END-OF-WEEK DELIVERABLE:** WEEK 7: App live on Railway + Vercel. Gmail OAuth working on production domain. README complete. Demo video recorded. Production smoke test passed.

WEEK 8
Jun 5–6

PHASE 8 — Final Buffer & Submission

▸ FRIDAY *June 5*

- **Final review and submission package assembly**
  Verify: GitHub repo is public, production URL is live and responsive (Railway free plan may sleep — disable sleep for submission period), README is accurate, demo video is accessible. Assemble the submission package per your professor's requirements.
- **One final end-to-end check**
  Log in on production, submit one deliverable, confirm the email arrives. Confirm the professor dashboard shows it. That's the proof it all works.

▸ **SATURDAY** *June 6 — DEADLINE*

- **SUBMIT**
  Submit before the deadline. Confirm receipt. Done.

✓ **END-OF-WEEK DELIVERABLE:** PROJECT DELIVERED: Full-stack AI-powered platform with real Gmail integration. Students write in the platform, receive confirmation and feedback emails. Professors get notifications, manage Gmail accounts via Settings, and override AI scores.

APPENDIX A — GMAIL API SETUP STEP-BY-STEP

Follow these steps exactly in Google Cloud Console before writing any code:

Step 1	Go to console.cloud.google.com and create a new project called 'SE Autograder'.
Step 2	Enable the Gmail API: APIs & Services → Library → search 'Gmail API' → Enable.
Step 3	Create OAuth credentials: APIs & Services → Credentials → Create Credentials → OAuth 2.0 Client ID → Web Application.
Step 4	Add Authorized Redirect URIs: http://localhost:8000/auth/gmail/callback (dev) AND https://your-backend.railway.app/auth/gmail/callback (prod).
Step 5	Add Authorized JavaScript Origins: http://localhost:5173 (dev) AND https://your-app.vercel.app (prod).
Step 6	Download the credentials JSON. Copy client_id and client_secret to your .env as GOOGLE_CLIENT_ID and GOOGLE_CLIENT_SECRET.
Step 7	Configure OAuth consent screen: set app name 'SE Autograder', add your email as test user. Keep status 'Testing' for the project.
Step 8	Generate a Fernet key for encrypting stored tokens: python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())" and add to .env as FERNET_KEY.

APPENDIX B — COMMAND CHEAT SHEET

Action	Command
Start backend	uvicorn app.main:app --reload
Start frontend	npm run dev
Run tests	pytest -v
Tests + coverage	pytest --cov=app tests/
New migration	alembic revision --autogenerate -m 'desc'
Apply migrations	alembic upgrade head

Generate Fernet key	python -c "from cryptography.fernet import Fernet; print(Fernet.generate_key().decode())"
Seed database	python scripts/seed.py
Test Gmail send	python -c "from app.services.email_service import send_email; ..."
Daily commit	git add . && git commit -m 'feat: ...' && git push